Institute of Management Sciences
Peshawar

Assignment

Parallel sorting

SWE 537 — Parallel and Distributed Computing (Lab)

Lab Instructor: Saim Sanan

BS Artificial Intelligence, BS Software Engineering Fall 2026

Task

On the Ubuntu virtual machine, select one sorting algorithm. Measure the serial execution time. Construct a parallel version. Measure the parallel execution time. Account for the results from the recorded measurements.

No parallel program is supplied. The written account must follow from the recorded execution.

1 Programs

Prepare three C source files.

1. `serial.c` performs the sort as a serial portion only. It contains no parallel construct.
2. `attempt.c` retains the first parallel version in the state in which the sorted result was incorrect, or the execution time exceeded the serial execution time. Do not modify this file after that execution.
3. `parallel.c` is the corrected version. Its sorted output must be identical to that of `serial.c`.

All three programs must compile and run on the Ubuntu virtual machine.

When an execution time is under 0.2 seconds, increase the input size and repeat all three executions. Use the same input size in all three.

The input depends on the roll number, so that one submission cannot be substituted for another. Use the roll number as the random seed. Generate the same unsorted data in each program.

Each program reports, in this order:

- the roll number
- the input size
- `serial`, `attempt`, or `parallel`
- one execution time in seconds, taken by a wall-clock timer around the sort only
- the first value and the last value of the sorted sequence

`parallel.c` also reports the number of threads that took part in the parallel region. This is the team size during that region, not the value reported by `nproc`.

`serial.c` writes the complete sorted sequence to `serial_out.txt`. `parallel.c` writes the complete sorted sequence to `parallel_out.txt`. Each file contains the full sequence, one value after another, with nothing omitted. The command `diff serial_out.txt parallel_out.txt` produces no output when the sequences match.

2 Report (one page)

Each numerical value in the report must appear in the recorded output.

1. Include the `lscpu` fields CPU(s), Core(s) per socket, and Thread(s) per core, together with the output of `nproc`.
2. Include the recorded output of all three programs, and the result of `diff serial_out.txt parallel_out.txt`.
3. Identify the parallelized portion of the algorithm and the serial portion.

4. Compute the speedup from the serial execution time and the `parallel.c` execution time. Show the division.

5. The speedup need not equal `nproc`. Using the recorded execution times and the recorded processor information, account for the speedup obtained.

6. For `attempt.c`, state the defect in the result or in the execution time, and state the correction made in `parallel.c`.

7. State one further optimization that was not implemented. State the expected effect on execution time, and state what could invalidate that expectation on the machine used.

3 Submit

Submit one zip archive containing `serial.c`, `attempt.c`, `parallel.c`, and the one-page report.

A complete submission is one in which the three programs compile, every reported value appears in the recorded output, and `diff serial_out.txt parallel_out.txt` produces no output.